CHAPTER 02: TESTING THROUGHOUT THE SDLC (මෘදුකාංග සංවර්ධන ජීවන චක්‍රය තුළ පරීක්ෂාව)

1. Core Syllabus Concepts & In-Depth Notes (ප්‍රධාන විෂය කරුණු විස්තරාත්මකව)

2.1 Testing in the Context of an SDLC FL-2.1.1

> **Formal Definition: SDLC (Software Development Lifecycle) model** එකක් යනු මෘදුකාංගයක් නිර්මාණය කිරීමේදී අනුගමනය කරන විවිධ අදියරයන් (phases) අතර පවතින තාර්කික (logical) සහ කාලානුක්‍රමික (chronological) සබඳතාවය විදහා දක්වන වියුක්ත රාමුවකි.

SDLC Models සහ Testing සිදුවන ආකාරය:

1. Sequential Development Models (අනුක්‍රමික මාදිලි):

Waterfall සහ V-Model: සෑම අදියරක්ම අවසන් වූ පසු ඊළඟ අදියර ආරම්භ වේ. Dynamic testing ක්‍රියාත්මක වන්නේ development අදියර අවසන් වී executable code ලැබුණු පසුවය. Requirements වෙනස් නොවන ස්ථාවර ව්‍යාපෘති සඳහා සුදුසුය.

2. Iterative & Incremental Models (පුනරාවර්තන මාදිලි):

Agile, Scrum, Kanban, Extreme Programming: මෘදුකාංගය කුඩා කොටස් (iterations/sprints) ලෙස සංවර්ධනය කෙරේ. සෑම iteration එකක් අවසානයේම ක්‍රියාත්මක වන increment එකක් ලැබේ. සෑම iteration එකකදීම dynamic testing සෑම test level එකකදීම සිදුවේ.

Universal Good Testing Practices across ALL SDLCs (සෑම SDLC එකකටම පොදු යහපත් පිළිවෙත් 4) FL-2.1.2

1. **Corresponding Test Activity:** සෑම මෘදුකාංග සංවර්ධන ක්‍රියාකාරකමකටම අනුරූප වූ පරීක්ෂණ ක්‍රියාකාරකමක් තිබිය යුතුය (For every development activity, there is a corresponding test activity).
2. **Specific Test Objectives:** සෑම test level එකකටම එකිනෙකට වෙනස්, නිශ්චිත අරමුණු (distinct objectives) තිබිය යුතුය.
3. **Early Analysis and Design:** Test analysis සහ test design අදාළ සංවර්ධන අදියර ක්‍රියාත්මක වන අතරතුරදීම (කේත ලිවීමට පෙර) ආරම්භ විය යුතුය.

4. **Early Involvement in Reviews:** පද්ධතියේ specifications හා requirements සමාලෝචනය කිරීමට (static testing) testers ලා මුල් අවධියේදීම සහභාගී විය යුතුය.

2.1.3 Test-First Approaches (පරීක්ෂණ-පෙරමුණු ප්‍රවේශයන්) FL-2.1.3 (K1)

කේත ලිවීමට *පෙර* test cases ලිවීම මගින් දෝෂ වළක්වා ගැනීමේ ප්‍රවේශයන් 3කි:

TDD (Test-Driven Development):

Kent Beck විසින් හඳුන්වා දෙන ලදී. Developer මුලින්ම fail වන automated unit test එකක් ලියයි. ඉන්පසු එය pass වීමට පමණක් ප්‍රමාණවත් code ලියයි. අවසානයේ code එක refactor කරයි (Red $\rightarrow$ Green $\rightarrow$ Refactor).

ATDD (Acceptance TDD):

Markus Gärtner විසින් විස්තර කරන ලදී. කණ්ඩායම (Developers, Testers, Business Reps) එකතු වී user stories වල Acceptance Criteria මත පදනම්ව acceptance tests සාමූහිකව සැලසුම් කරයි.

BDD (Behavior-Driven Development):

Dan North / David Chelimsky. පද්ධතියේ හැසිරීම සාමාන්‍ය ස්වාභාවික භාෂාවෙන් (Natural Language) `Given / When / Then` ආකෘතිය භාවිතයෙන් ලියනු ලැබේ (Gherkin syntax).

DevOps & Testing FL-2.1.4 :

Development සහ Operations ඒකාබද්ධ කර CI/CD (Continuous Integration / Continuous Delivery) පයිප්පලය හරහා කේතය commit කළ වහාම automated regression tests ක්‍රියාත්මක කර ක්ෂණික ප්‍රතිපෝෂණ (fast feedback) ලබාදෙයි.

Shift Left Concept FL-2.1.5 :

පරීක්ෂණ ක්‍රියාකාරකම් කාලරේඛාවේ වමට (මුල් අවධියට) ගෙන ඒම. උදාහරණ: Requirements විමර්ශනය කිරීම, කේත ලිවීමට පෙර test cases හැදීම, මුල් අවධියේ static analysis සිදුකිරීම.

2.1.6 Retrospectives and Process Improvement (ක්‍රියාවලි වැඩිදියුණු කිරීම සඳහා Retrospectives) FL-2.1.6 (K2)

Retrospectives (පසු විපරම් සාකච්ඡා):

සෑම Iteration (Sprint), Milestone, හෝ Release එකක් අවසානයේ මුළු කණ්ඩායමම එකතු වී පවත්වන සාකච්ඡාවකි. එහිදී:

- **1. සාර්ථක වූ කරුණු:** හොඳින් ක්‍රියාත්මක වූ දෑ හඳුනා ගැනීම (What went well).
- **2. අඩුපාඩු / ගැටලු:** වැඩිදියුණු විය යුතු අංශ සාකච්ඡා කිරීම (What could be improved).
- **3. ඉදිරි ක්‍රියාමාර්ග:** ඊළඟ iteration එක සඳහා වැඩිදියුණු කිරීම් ක්‍රියාත්මක කරන ආකාරය (Action items).

Testing සඳහා ප්‍රතිලාභ: Test efficiency ඉහළ යාම, Testware වල ගුණාත්මක බව වැඩිවීම, Defect root causes හඳුනාගෙන අනාගතයේදී දෝෂ වැළැක්වීම, සහ කණ්ඩායමේ සන්නිවේදනය දියුණු වීම. (*සටහන:* v4.0.1 errata අනුව පැරණි "post-project meetings" යන වචනය ඉවත් කර ඇත).

1.2 The Five Test Levels and Four Test Types (පරීක්ෂණ මට්ටම් 5 සහ වර්ග 4)

FL-2.2

The 5 Test Levels (පරීක්ෂණ මට්ටම් 5):

Test Level	පරීක්ෂණ ඉලක්කය (Test Object)	ප්‍රධාන අවධානය (Focus & Roles)
1. Component Testing (Unit Testing)	තනි මොඩියුල, classes, functions, code units.	ස්වාධීනව ක්‍රියාත්මක වන කුඩා කේත කොටස්වල දෝෂ සෙවීම. සාමාන්‍යයෙන් Developers ලා විසින් සිදුකරයි.
2. Component Integration Testing	මොඩියුල දෙකක් හෝ වැඩි ගණනක් අතර අතුරුමුහුණත් (interfaces) සහ interactions.	කේත කොටස් අතර දත්ත හුවමාරුව සහ communication පරීක්ෂා කිරීම. Developers හෝ Testers.
3. System Testing	සමස්ත පද්ධතියම (Integrated system as a whole).	End-to-end functional සහ non-functional (performance, security) අවශ්‍යතා සපුරා ඇත්දැයි බැලීම. Independent Testers.
4. System Integration Testing	අපේ පද්ධතිය සහ බාහිර පද්ධති, external APIs, cloud services අතර අතුරුමුහුණත්.	පිටස්තර බැංකු ගෙවීම් පද්ධති (Payment Gateways), 3rd party microservices සමග සබඳතා.
5. Acceptance Testing	සමස්ත නිමි පද්ධතිය.	පද්ධතිය deploy කිරීමට සූදානම්ද, ගිවිසුම්ගත අවශ්‍යතා සපුරා ඇත්දැයි තහවුරු කිරීම. (UAT, Operational, Alpha/Beta).

The 4 Test Types (පරීක්ෂණ වර්ග 4 - සියලුම මට්ටම් සඳහා අදාළ වේ):

විභාග රීතිය: CTFL v4.0.1 ට අනුව මෙම පරීක්ෂණ වර්ග 4ම ඉහත සඳහන් කළ සියලුම Test Levels 5 තුළදීම ක්‍රියාත්මක කළ හැක:

1. Functional Testing:
පද්ධතිය කුමක් කරන්නේද (**WHAT the system does**) යන්න පරීක්ෂා කරයි. Business requirements, user stories, functions වල නිවැරදිභාවය.

2. Non-Functional Testing:
පද්ධතිය කෙතරම් හොඳින් හැසිරෙන්නේද (**HOW WELL it behaves**) යන්න පරීක්ෂා කරයි. ISO/IEC 25010 ලක්ෂණ: Performance, Reliability, Security, Usability.

3. Black-Box Testing:
අභ්‍යන්තර කේත ව්‍යුහය නොදැන (specification-based) බාහිරින් inputs ලබාදී outputs පරීක්ෂා කිරීම.

4. White-Box Testing:
අභ්‍යන්තර source code, control flow, statements සහ branches පදනම් කරගෙන (structure-based) පරීක්ෂා කිරීම.

Confirmation Testing vs. Regression Testing FL-2.2.3 **:**

පරාමිතිය	**Confirmation Testing (Re-testing)**	**Regression Testing**
ප්‍රධාන අරමුණ	වාර්තා කළ දෝෂය (defect) නිවැරදිව fix කර ඇත්දැයි තහවුරු කිරීම.	සිදුකළ වෙනස්කම නිසා වෙනස් නොකළ අනෙකුත් කොටස් වලට අතුරු ආබාධ (side-effects) ඇති වී ඇත්දැයි බැලීම.
ක්‍රියාත්මක කරන පරාසය	කලින් fail වූ test case එක පමණක් run කරයි.	පද්ධතිය පුරා විහිදුනු regression test suite එකක් run කරයි.
Automation යෝග්‍යතාව	බොහෝවිට manual හෝ script එකකින් තහවුරු කරයි.	**නිරන්තරයෙන්ම automated** විය යුතුය (නැවත නැවත run වන බැවින්).

2.3 Maintenance Testing (නඩත්තු පරීක්ෂාව) FL-2.3.1

පද්ධතියක් production පරිසරයට ගිය පසු (operational phase) සිදුකරන පරීක්ෂාවයි. ප්‍රධාන Triggers (හේතු) 3කි:

- **1. Modifications (වෙනස්කම්):** Planned enhancements, bug fixes, operational emergency hotfixes.

- **2. Upgrades & Migrations (පරිසර සංක්‍රමණය):** On-premise සිට Cloud වෙත මාරුවීම, Database එක Oracle සිට PostgreSQL වෙත මාරුවීම (Data conversion testing අත්‍යවශ්‍ය වේ).
- **3. Retirement (විශ්‍රාම ගැන්වීම / ඉවත් කිරීම):** පද්ධතියක් භාවිතයෙන් ඉවත් කිරීමේදී දත්ත ආරක්ෂිතව archive කිරීම සහ නීතිමය කාලසීමාවන් තුළ ප්‍රතිසාධනය කළ හැකිදැයි පරීක්ෂා කිරීම.

Impact Analysis (බලපෑම් විශ්ලේෂණය): පද්ධතියේ සිදුකරන වෙනස්කමකින් බලපෑමට ලක්විය හැකි අනෙකුත් මොඩියුල හඳුනා ගැනීම. Maintenance testing හි විෂය පථය (scope) සහ අවශ්‍ය regression testing ප්‍රමාණය තීරණය කරන්නේ Impact Analysis මගිනි.

STUDENT NOTES & EXAM FLASHCARD REMINDERS (සිසු සටහන් සඳහා ඉඩ)

4. Realistic ISTQB Practice Exam Questions & Detailed Rationales (අභ්‍යාස ප්‍රශ්න සහ විවරණ)

Question 1 [K2 - FL-2.1.1]: How does testing in an Agile development model typically differ from testing in a sequential V-model?

(Agile සංවර්ධන ආකෘතියක පරීක්ෂාව සහ අනුක්‍රමික V-ආකෘතියක (Sequential V-Model) පරීක්ෂාව අතර ඇති ප්‍රධාන වෙනස කුමක්ද?)

A) Agile testing produces more comprehensive test documentation than sequential testing.

B) In Agile, dynamic testing is executed at all test levels in each iteration, whereas in sequential models, dynamic testing occurs mainly in later phases.

C) Agile projects do not require regression testing, whereas sequential projects automate all regression tests.

D) In sequential models, testers perform only non-functional testing, while developers perform functional testing.

Correct Answer: B

Rationale (විවරණය): Agile වලදී සෑම iteration එකකදීම working software increment එකක් ලැබෙන බැවින්, සෑම iteration එකකදීම සියලු test levels වල dynamic testing සිදුවේ. Sequential (V-model) වලදී dynamic testing සිදුවන්නේ අවසන් අදියර වලදීය.

Question 2 [K1 - FL-2.1.2]: Which of the following is a recognized GOOD TESTING PRACTICE that applies to ALL software development lifecycles?

(සියලුම මෘදුකාංග සංවර්ධන ජීවන චක්‍රයන්ට (SDLCs) අදාළ වන පිළිගත් යහපත් පරීක්ෂණ පිළිවෙතක් (Good Testing Practice) වන්නේ කුමක්ද?)

A) Test execution must be completely automated before coding starts.

B) Different test levels must have identical test objectives to ensure complete overlap.

C) For every software development activity, there is a corresponding test activity.

D) Non-functional testing should only be performed during User Acceptance Testing.

Correct Answer: C

Rationale (විවරණය): Section 2.1.2 ට අනුව ඕනෑම SDLC එකකට පොදු මූලික යහපත් පිළිවෙත වන්නේ: "For every software development activity, there is a corresponding test activity" යන්නයි.

Question 3 [K1 - FL-2.1.3]: In which test-first approach are test cases written in a simple natural language using the Given/When/Then format?

(Given/When/Then ආකෘතිය භාවිතයෙන් සරල ස්වභාවික භාෂාවෙන් Test cases ලියනු ලබන test-first ප්‍රවේශය කුමක්ද?)

A) Test-Driven Development (TDD)

B) Behavior-Driven Development (BDD)

C) Acceptance Test-Driven Development (ATDD)

D) Feature-Driven Development (FDD)

Correct Answer: B

Rationale (විවරණය): Natural language සහ Given/When/Then ආකෘතිය භාවිතයෙන් පද්ධතියේ හැසිරීම විස්තර කරන්නේ **BDD (Behavior-Driven Development)** වලදීය.

Question 4 [K2 - FL-2.1.5]: Which of the following actions is an example of applying the "SHIFT LEFT" approach in testing?

(පරීක්ෂාවේදී "SHIFT LEFT" ප්‍රවේශය භාවිත කිරීමට උදාහරණයක් වන්නේ පහත කුමන ක්‍රියාවද?)

A) Postponing performance testing until the entire system is deployed to production.

B) Reviewing requirements and user stories from a tester's perspective before code implementation begins.

C) Replacing all automated regression testing with manual user acceptance testing.

D) Having developers execute system integration testing without requirements.

Correct Answer: B

Rationale (විවරණය): කේත ලිවීමට පෙර specifications සහ user stories පරීක්ෂකයාගේ ඇසින් සමාලෝචනය කිරීම (early static testing) යනු Shift Left ප්‍රවේශයේ ප්‍රධාන උදාහරණයකි.

Question 5 [K2 - FL-2.2.1]: Which test level focuses specifically on testing the interactions between the system under test and other external systems and cloud services?

(පරීක්ෂාවට ලක්වන පද්ධතිය සහ බාහිර පද්ධති හෝ cloud සේවාවන් අතර අන්තර්ක්‍රියා පරීක්ෂා කිරීම කෙරෙහි විශේෂයෙන් අවධානය යොමු කරන පරීක්ෂණ මට්ටම (Test Level) කුමක්ද?)

A) Component Testing

B) Component Integration Testing

C) System Integration Testing

D) User Acceptance Testing

Correct Answer: C

Rationale (විවරණය): අපේ පද්ධතිය සහ බාහිර පද්ධති (external systems, cloud services, 3rd party APIs) අතර interactions පරීක්ෂා කරන්නේ **System Integration Testing** මට්ටමේදීය. Component integration පරීක්ෂා කරන්නේ internal components අතරය.

Question 6 [K2 - FL-2.2.2]: Which of the following statements about test types is TRUE?

(පරීක්ෂණ වර්ග (Test Types) පිළිබඳ පහත සඳහන් ප්‍රකාශ අතුරින් සත්‍ය වන්නේ කුමක්ද?)

A) Functional testing evaluates how well the system behaves, while non-functional testing evaluates what it does.

B) Non-functional testing can only be conducted during system testing.

C) Black-box, white-box, functional, and non-functional testing can all be applied at all test levels.

D) White-box testing derives test cases from user stories and business requirements.

Correct Answer: C

Rationale (විවරණය): Section 2.2.2 ට අනුව test types 4ම (functional, non-functional, black-box, white-box) සියලුම test levels 5 තුළදීම ක්‍රියාත්මක කළ හැක.

Question 7 [K2 - FL-2.2.3]: A tester executes a test case that previously failed to confirm that a bug fix resolved the issue. Simultaneously, an automated test suite executes across the entire application to ensure unchanged modules were not corrupted. What types of testing are being performed?

(දෝෂ නිවැරදි කිරීමක් තහවුරු කිරීමට ටෙස්ටර් කෙනෙක් කලින් අසමත් වූ test case එකක් නැවත ක්‍රියාත්මක කරයි. ඒ සමඟම වෙනස් නොකළ කොටස්වලට හානියක් වී නැති බව තහවුරු කිරීමට automated test suite එකක් ක්‍රියාත්මක කරයි. මෙහිදී සිදුකරන පරීක්ෂණ වර්ග මොනවාද?)

A) Regression testing followed by maintenance testing.

B) Confirmation testing and regression testing, respectively.

C) System integration testing and alpha testing.

D) Non-functional testing and confirmation testing.

Correct Answer: B

Rationale (විවරණය): නිවැරදි කළ bug එක නැවත බැලීම **Confirmation Testing (re-testing)** වන අතර, වෙනස් නොකළ කොටස් වලට හානි වී ඇත්දැයි බැලීම **Regression Testing** වේ.

Question 8 [K2 - FL-2.3.1]: An enterprise migrates its customer database from an on-premise Oracle server to an AWS cloud-hosted PostgreSQL database. Which type of testing is triggered, and what is its primary focus?

(ආයතනයක් තම පාරිභෝගික දත්ත ගබඩාව On-premise Oracle සේවාදායකයක සිට AWS Cloud PostgreSQL දත්ත ගබඩාවකට සංක්‍රමණය (migrate) කරයි. මෙහිදී ආරම්භ වන පරීක්ෂණ වර්ගය සහ එහි ප්‍රධාන අවධානය කුමක්ද?)

A) Component testing to rewrite database algorithms.

B) Maintenance testing triggered by an upgrade/migration, focusing on tests of the new environment and data conversion.

C) Retirement testing to delete legacy customer data permanently.

D) System integration testing focused exclusively on user interface styling.

Correct Answer: B

Rationale (විවරණය): Database එකක් cloud එකට migrate කිරීම යනු Maintenance Testing සඳහා trigger එකකි. මෙහිදී නව පරිසරය සහ දත්ත පරිවර්තනය (data conversion testing) ප්‍රධාන වශයෙන් පරීක්ෂා කෙරේ.

Part 2: Additional Questions for Remaining Learning Objectives (ඉතිරි ඉගෙනුම් අරමුණු සඳහා අමතර ප්‍රශ්න)

Question 9 [K2 - FL-2.1.4]: Which of the following is a typical impact of DevOps on testing?

(DevOps මගින් පරීක්ෂාවට ඇතිවන සාමාන්‍ය බලපෑමක් කුමක්ද?)

A) Automated regression tests in the CI/CD pipeline give fast feedback on quality and reduce the risk of regression defects.

B) Because deployment is continuous, non-functional characteristics no longer need to be tested.

C) Test automation is no longer necessary because the pipeline tests itself.

D) Testing is moved to the very end of the pipeline so that development is never slowed down.

Correct Answer: A

Rationale (සිංහල + English):

නිවැරදි පිළිතුර A. DevOps හි වාසි: quality පිළිබඳ ඉක්මන් ප්‍රතිපෝෂණ, CI මගින් shift-left ප්‍රවර්ධනය, CI/CD මගින් ස්ථායී පරීක්ෂණ පරිසර, non-functional ලක්ෂණ (performance, reliability) කෙරෙහි වැඩි අවධානය, ස්වයංක්‍රීයකරණයෙන් නැවත නැවත කරන manual පරීක්ෂණ අඩුවීම සහ regression අවදානම අඩුවීම. අවදානම්: delivery pipeline එක නිර්වචනය/ස්ථාපනය කළ යුතුය; CI/CD මෙවලම් හඳුන්වාදී පවත්වාගත යුතුය; test automation සඳහා අමතර සම්පත් අවශ්‍ය වන අතර එය ස්ථාපනය/පවත්වාගැනීම අපහසු විය හැකිය.

English: DevOps benefits for testing: fast feedback on quality, CI promoting shift-left, stable test environments through CI/CD automation, more focus on non-functional characteristics (e.g. performance, reliability), less repetitive manual testing and a lower regression risk. Risks: the delivery pipeline must be defined and established, CI/CD tools must be introduced and maintained, and test automation needs extra resources and can be hard to build and maintain (Section 2.1.4).

Question 10 [K2 - FL-2.1.6]: How can retrospectives be used as a mechanism for process improvement?

(ක්‍රියාවලි වැඩිදියුණු කිරීමේ යාන්ත්‍රණයක් ලෙස retrospectives භාවිත වන්නේ කෙසේද?)

A) Management alone holds them before the project starts in order to assign blame for earlier failures.

B) They automatically re-run all regression tests to compare the results of two iterations.

C) The team discusses what was successful and what could be improved, agrees how to incorporate improvements, and records the results (for example in the test completion report).

D) They replace test completion reports and are never documented.

Correct Answer: C

Rationale (සිංහල + English):

නිවැරදි පිළිතුර C. Retrospective එක ව්‍යාපෘතියක/iteration එකක අවසානයේ හෝ release milestone එකකදී පැවැත්වේ. කණ්ඩායම ඵලදායී වූ දේ (පවත්වාගත යුතු) සහ වැඩිදියුණු කළ යුතු දේ සාකච්ඡා කර, වැඩිදියුණු කිරීම් ක්‍රියාත්මක කරන ආකාරය තීරණය කරයි; ප්‍රතිඵල වාර්තා කර (උදා: test completion report එකේ) තබාගනී. ප්‍රතිලාභ: පරීක්ෂණ effectiveness/efficiency වැඩිවීම, testware ගුණාත්මකභාවය, කණ්ඩායම් බැඳීම සහ ඉගෙනීම, test basis ගුණාත්මකභාවය සහ සංවර්ධනය-පරීක්ෂණ සහයෝගීතාව.

English: Retrospectives are held at the end of a project or iteration, at a release milestone, or when needed. Participants discuss what to retain, what to improve and how to incorporate improvements; the results should be recorded, typically in the test completion report. Benefits for testing: higher test effectiveness and efficiency, better testware quality, team bonding and learning, better test basis quality and better cooperation between development and testing (Section 2.1.6).